
Systemes Embarqués



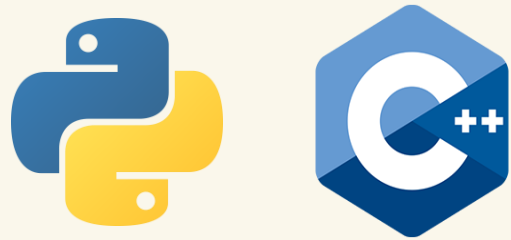
ROB 2A, UE 4.1, ENSTA

Cross-compilation

SYSTÈMES EMBARQUÉS

Objectif

Déployer des programmes intelligents sur des plateformes embarquées (ordinateurs, microcontrôleurs)



ROS



Système temps réel

Définition : **Système temps réel**

Système capable de contrôler ou réagir à un procédé physique à une vitesse adaptée à l'évolution du procédé contrôlé

En **robotique autonome** la latence, la prédictibilité et la réactivité sont critiques.

Exemple :

un robot mobile utilise des LiDAR ou des caméras pour détecter et éviter des obstacles en mouvement (temps de réaction de l'ordre de 50, 100, 500 ms ?)

-> afin d'être efficace, il faut perdre le moins temps possible entre et pendant **la mesure, le filtrage, la prise de décision et l'actionnement**

Ressources limitées

Contrainte :

Les systèmes embarqués (robots, drones, microcontrôleurs) ont des **ressources limitées** (CPU, RAM, ...)

=> Choix de systèmes d'exploitation temps réels **RTOS** (FreeRTOS, Zephyr, ...) pour les microcontrôleurs (Arduino, ESP32, STM32)

=> Choix des **langages compilés** (C/C++, ...) plutôt qu'interprétés (python, JavaScript, ...)

ROS2 + Raspberry Pi et temps réel

Focus sur des applications ROS2 sur ordinateur embarqué (type Raspberry Pi)

Utilisation de **Ubuntu** ou **Raspberry Pi OS** souvent préférée, mais ce ne sont **pas des RTOS**

=> un tel système **ne peut pas garantir du temps réel strict** (hard real-time), mais peut s'en approcher pour des applications **temps réel souple** (soft real-time) si on l'optimise

Remarque : pour des applications temps réel strict **combiner Raspberry Pi (haute couche) + microcontrôleur (temps réel)** est souvent la meilleure solution

Langages compilés VS interprétés

Pourquoi choisir des langages compilés (C, C++, Rust) pour les applications embarquées ?

1. **Fichiers compilés optimisés et spécifiques à l'architecture cible**
2. **Rapide** : exécution directe, pas de surcharge de l'interpréteur
3. **Faible empreinte mémoire** : Les binaires compilés sont **plus légers** que les environnements d'exécution
4. **Accès direct au matériel** : registres, mémoire

=> Idéal pour les systèmes embarqués temps réel

Problème de portabilité

Langage interprété :

Portable (le code source fonctionne partout où l'interpréteur est installé)

Langage compilé :

Dépend de l'architecture de l'environnement de compilation

Les systèmes embarqués utilisent souvent des **processeurs ARM** (ex: Raspberry Pi, Jetson, STM32), tandis que nos machines de développement sont généralement des **x86_64** (PC Intel/AMD)

Un binaire compilé pour x86 **ne fonctionnera pas** sur ARM, et vice versa

Limites de la compilation native

Pourquoi ne pas compiler directement sur la cible ?

- **Ressources limitées** : Compiler directement sur un Raspberry Pi ou un Jetson est lent ou impossible, surtout pour des projets ROS 2 complexes
- **Manque de flexibilité** :
 - Difficile de maintenir plusieurs environnements de développement
 - Dans le cas d'une flotte, besoin de recompiler individuellement sur chaque robot

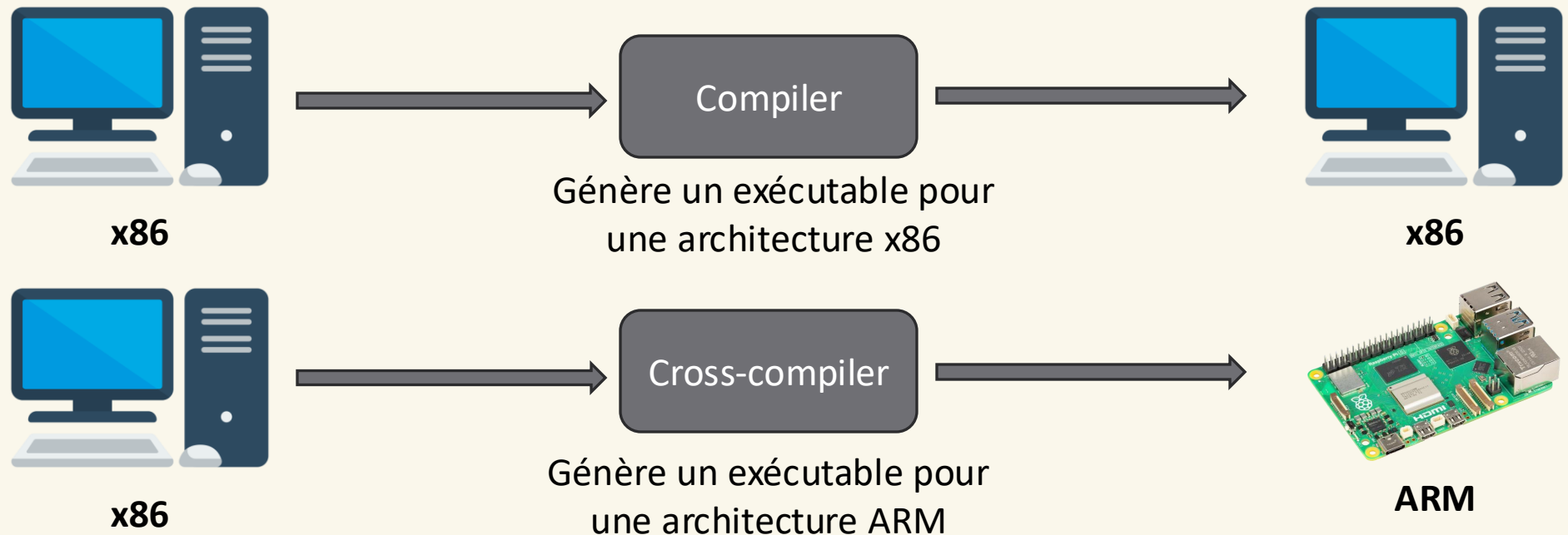
Solution ?

Comment développer et tester efficacement du code sur une machine de développement (ex: mon PC x86_64) quand la cible finale est une Raspberry Pi, un NVIDIA Jetson, ou un microcontrôleur ARM ?

La réponse : **la cross-compilation**

La cross-compilation

La **cross-compilation** est le processus de compilation d'un programme sur une machine (appelée *machine hôte*) pour qu'il s'exécute sur une autre machine (appelée *machine cible*), dont l'architecture matérielle est différente.



Docker – Multiplatform images



Une **image multi-plateforme** peut être exécutée sur plusieurs plateformes telles que [linux/amd64](#), [linux/arm64](#), et [windows/amd64](#)

<https://docs.docker.com/build/building/multi-platform/>

Elle permet de compiler du code pour ARM64 dans un container Docker sur un PC x86, lequel fournit un environnement de compilation isolé de votre machine hôte, facilement répétable et partageable.